

Sunum Notları · Bölüm 3: Hazır Bir Modeli İnce Ayarlama (Fine-tuning)

İçindekiler

1. 3/1 · Giriş
2. 3/2 · Veri İşleme: Hugging Face ile BERT ve MRPC Örneği
3. 3/3 · Trainer API ile model eğitimi
4. 3/4 · Tam eğitim döngüsü ve Accelerate
5. 3/5 · Öğrenme eğrilerini anlama
6. 3/6 · Fine-tuning, Check! (Bölüm özeti)

3/1 · Giriş

"Geçtiğimiz bölümde hazır yapay zeka modellerini nasıl alıp kullanacağımızı işlemiştik. Bu bölümde ise çok daha heyecan verici bir aşamaya geçiyoruz: Hazır bir modeli, kendi özel problemimizi çözecek şekilde nasıl yeniden eğitiriz (fine-tuning)?"

Eğitimde Neler Öğreneceğiz?

Sunumu 3 ana bölüme ayırabiliriz:

- **Veri Hazırlığı (Bölüm 2):** Büyük veri setlerini internetten çekip, yapay zeka eğitimine en uygun şekilde nasıl hazırlayacağımızı göreceğiz.
- **Hızlı ve Pratik Eğitim (Bölüm 3):** 'Trainer' adını verdiğimiz modern araçları kullanarak, bir modeli en iyi pratiklerle nasıl kolayca eğitebileceğimizi keşfedeceğiz.
- **Kontrolü Ele Alma ve Hızlandırma (Bölüm 4):** İhtiyaç duyduğumuzda eğitim sürecini sıfırdan nasıl kodlayacağımızı ve Accelerate adlı aracı kullanarak ağır eğitim işlemlerini farklı cihazlara (örneğin birden fazla ekran kartına) nasıl dağıtacağımızı öğreneceğiz.

Kullanacağımız Araçlar

Eğitim boyunca sektörün standartı haline gelen PyTorch altyapısını kullanacağız. Ayrıca Hugging Face'in sadece ana model kütüphanesini değil; veri işleme, değerlendirme ve sistemi hızlandırma gibi işleri kolaylaştıran yan araçlarını da (Datasets, Tokenizers, Evaluate) tanımış olacağız.

Günün Sonunda Ne Elde Edeceğiz?

Tüm bu adımları tamamladığımızda, popüler bir dil modeli olan BERT'i, metin sınıflandırma görevi için bizzat eğitmiş olacağız. Üstelik kendi eğittiğiniz bu modelleri tüm dünyayla paylaşmak için tek ihtiyacınız olan şey ücretsiz bir Hugging Face hesabı açmak.

3/2 · Veri İşleme: Hugging Face ile BERT ve MRPC Örneği

0. Önce Bu Kavramları Bil

Kavram	Anlamı
Tokenizer	Metni modelin anlayacağı sayılara çeviren araç
Token	Metnin küçük parçası (kelime ya da kelime parçası)
Batch	Modele aynı anda verilen örnek grubu
Padding	Kısa cümleleri özel bir "boş" token ile uzun olana eşitleme
Truncation	Çok uzun cümleleri kesme
Checkpoint	Hazır, önceden eğitilmiş model (örn. bert-base-uncased)
Fine-tuning	Hazır modeli kendi verimizle ek eğitme
Label (etiket)	Verinin doğru cevabı (örn. 0 veya 1)
Loss	Modelin ne kadar yanlış yaptığının ölçüsü
Hub	Hugging Face'in model ve veri seti deposu

1. Bölümün Amacı

Önceki bölümde modeli sadece 2 cümleyle eğitmiştik. Bu iyi sonuç vermez. Bu bölümde gerçek bir veri seti hazırlamayı öğreniyoruz.

Önceki bölümden örnek (tek batch eğitimi):

```
import torch
from torch.optim import AdamW
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequences = [
    "I've been waiting for a HuggingFace course my whole life.",
    "This course is amazing!",
]
batch = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")

batch["labels"] = torch.tensor([1, 1]) # YENİ: doğru cevaplar

optimizer = AdamW(model.parameters()) # modeli güncelleyecek araç
loss = model(**batch).loss # hata hesapla
loss.backward() # hatanın nedenini geriye doğru hesapla
optimizer.step() # modeli güncelle
```

2. MRPC Veri Seti

- MRPC = Microsoft Research Paraphrase Corpus
- 5.801 cümle çifti içerir.
- Etiket: İki cümle aynı anlama geliyor mu? (paraphrase)
- GLUE benchmark'ının 10 görevinden biridir. GLUE, modelleri 10 farklı metin sınıflandırma görevinde ölçen akademik bir test setidir.
- Küçük olduğu için denemeye uygundur.

3. Veri Setini Hub'dan Yükleme

```
from datasets import load_dataset  
  
raw_datasets = load_dataset("glue", "mrpc")
```

"glue" = veri seti ailesi, "mrpc" = GLUE içindeki görev.

Sonuç bir DatasetDict'tir (3 bölümlü sözlük):

Bölüm	Satır	Ne için?
train	3.668	Eğitim
validation	408	Eğitim sırasında kontrol
test	1.725	Son değerlendirme

Sütunlar: sentence1, sentence2, label, idx

Veri ~/.cache/huggingface/datasets klasörüne indirilir (HF_HOME ile değiştirilebilir).

Örnek veriye bakma:

```
raw_train_dataset = raw_datasets["train"]  
raw_train_dataset[0]
```

```
{'idx': 0,  
 'label': 1,  
 'sentence1': 'Amrozi accused his brother , whom he called " the witness " , of deliberately distorting his evidence .',  
 'sentence2': 'Referring to him as only " the witness " , Amrozi accused his brother of deliberately distorting his evidence .'}  
}
```

Etiketler: Zaten sayısı olduğu için ek işlem gerekmez.

- 0 = not_equivalent (aynı anlamda değil)
- 1 = equivalent (aynı anlamda)

```
raw_train_dataset.features # etiket eşlemesini gösterir
```

4. Ön İşleme (Preprocessing)

Neden gerekli? Model metni değil sayıları anlar. Bu yüzden tokenizer kullanılır.

Sorun: İki cümleyi ayrı ayrı tokenize edersek model bunları çift olarak anlayamaz. Çift olarak verilmelidir:

```
inputs = tokenizer("This is the first sentence.", "This is the second one.")
```

Çıktı üç anahtar içerir:

Anahtar	Görevi
input_ids	Her token'ın sayısal karşılığı
attention_mask	1 = gerçek token, 0 = padding (yok say)
token_type_ids	Token hangi cümleye ait? (0 = 1. cümle, 1 = 2. cümle)

BERT'in beklediği biçim:

[CLS] cümle1 [SEP] cümle2 [SEP]

- [CLS] = başlangıç işareti

- [SEP] = cümleleri ayıran işaret

```
Token: [CLS] this is the first sentence . [SEP] this is the second one . [SEP]
token_type_ids: 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1
```

token_type_ids neden var?

BERT ön eğitimde Next Sentence Prediction görevini öğrenmiştir: "İkinci cümle, birincinin devamı mı?" Bu yüzden hangi cümlenin hangisi olduğunu bilmesi gerekir.

Not: Her modelde token_type_ids olmaz (örn. DistilBERT'te yok). Tokenizer ve model aynı checkpoint olduğu sürece tokenizer doğru girdiyi kendisi üretir, endişelenmeye gerek yok.

5. Tüm Veri Setini İşleme

Yöntem 1 (önerilmez): Tokenizer'a tüm veriyi birden vermek.

```
tokenized_dataset = tokenizer(
    raw_datasets["train"]["sentence1"],
    raw_datasets["train"]["sentence2"],
    padding=True,
    truncation=True,
)
```

Dezavantajları:

- Sonuç düz bir sözlüktür, veri seti yapısı kaybolur.
- Tüm veriyi RAM'e yükler, büyük veride sorun çıkar.

Yöntem 2 (önerilen): Dataset.map()

```
def tokenize_function(example):
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)

tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
```

- map() fonksiyonu her elemana verdiğimiz fonksiyonu uygular.
- Veri diskte (Apache Arrow formatında) kalır, RAM şişmez.
- batched=True → Örnekler toplu işlenir, çok daha hızlıdır (arkadaki Rust tabanlı tokenizer çok veriyle hızlı çalışır).
- Sonuçta veri setine yeni sütunlar eklenir: input_ids, attention_mask, token_type_ids.
- Padding burada bilinçli olarak yapılmadı (aşağıda açıklanıyor).
- İsteğe bağlı: num_proc ile çoklu işlemci kullanılabilir (hızlı tokenizer'da gerek yok).

6. Dinamik Padding

Problem: Cümleler farklı uzunlukta, ama bir batch içindeki örnekler aynı uzunlukta olmalı.

Yöntem	Açıklama
Sabit padding	Tüm veriyi en uzun örneğe göre doldur → çok boş token, yavaş
Dinamik padding ✓	Her batch'i sadece kendi içindeki en uzun örneğe göre doldur → hızlı

Not: TPU kullanılıyorsa sorun çıkabilir, çünkü TPU sabit boyut tercih eder.

Collate function: Örnekleri bir araya getirip batch yapan fonksiyondur. Padding'i bu yapar. Hugging Face bunun için hazır bir araç sunar:

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

Tokenizer'ı alır; hangi padding token'ını kullanacağını ve padding'in sağa mı sola mı yapılacağını bilir.

Deneme:

```
samples = tokenized_datasets["train"][:8]
samples = {k: v for k, v in samples.items() if k not in ["idx", "sentence1", "sentence2"]}
[ len(x) for x in samples["input_ids"] ]
[50, 59, 47, 67, 59, 50, 62, 32]
```

- İlk 8 örneğin uzunlukları 32 ile 67 arasında değişiyor.
- idx, sentence1, sentence2 sütunları silindi, çünkü içlerinde metin var ve metinden tensör (sayı matrisi) oluşturulamaz.

```
batch = data_collator(samples)
{k: v.shape for k, v in batch.items()}
{'attention_mask': torch.Size([8, 67]),
 'input_ids': torch.Size([8, 67]),
 'token_type_ids': torch.Size([8, 67]),
 'labels': torch.Size([8])}
```

- Hepsi batch içindeki en uzun örneğe göre 67'ye doldu.
- [8, 67] = 8 örnek × 67 token.
- label sütunu otomatik olarak labels adını aldı (modelin beklediği isim).

Artık ham metinden modelin işleyebileceği batch'lere ulaştık, fine-tuning'e hazırız.

7. Ana Çıkarımlar

- Dataset.map() + batched=True → ön işleme çok daha hızlı.
- Dinamik padding (DataCollatorWithPadding) → sabit padding'den daha verimli.
- Veri, modelin beklediği biçime getirilmeli: sayısal tensörler ve doğru sütun adları.
- Datasets kütüphanesi büyük veriyi verimli işlemek için güçlü araçlar sunar.

Sonraki adım: Trainer API ile modelin fine-tune edilmesi.

3/3 · Trainer API ile model eğitimi

Trainer, hazır bir modeli kendi verimizle eğitmemizi (fine-tuning yapmamızı) kolaylaştıran hazır bir sınıftır. Eğitim CPU'da çok yavaş olduğu için GPU gerekir; GPU'nuz yoksa Google Colab'dan ücretsiz GPU alabilirsiniz.

Önceki bölümde hazırladığımız şu nesneleri kullanacağız: **checkpoint**, tokenizer, **tokenized_datasets** ve **data_collator**.

Adım 1: TrainingArguments (Eğitim Argümanları)

TrainingArguments, eğitimin tüm ayarlarını tek yerde toplayan sınıftır. Zorunlu olan tek şey eğitilen modelin kaydedileceği klasörün adıdır; diğer ayarlar için varsayılan değerler yeterlidir.

```
from transformers import TrainingArguments
training_args = TrainingArguments("test-trainer")
```

Adım 2: Modeli tanımlama

Bu adımda, cümle çiftlerini iki gruptan birine (aynı anlamda / farklı anlamda) ayıracak modeli yüklüyoruz. Yüklerken çıkan uyarı normaldir: BERT'in eski head katmanı atılmış, yerine yeni ve rastgele başlatılmış bir katman eklenmiştir.

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint, num_labels=2)
```

Adım 3: Trainer'ı tanımlama

Hazırladığımız model, ayarlar ve veri setlerini Trainer'a veririz. Tokenizer'ı ise **processing_class** parametresiyle belirtiriz.

```
from transformers import Trainer

trainer = Trainer(
    model,
    training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    processing_class=tokenizer,
)
```

Adım 4: Eğitimi başlatma

train() komutu eğitimi başlatır, ancak ekranda yalnızca loss değerini gösterir. Modelin gerçekten ne kadar başarılı olduğunu görmek için bir evaluation ayarı ve bir metrik fonksiyonu eklememiz gerekir.

```
trainer.train()
```

Trainer

Eğitimi bizim yerimize yöneten hazır sınıf.

Hiperparametre

Eğitimden önce bizim seçtiğimiz ayar (learning rate, batch size).

Checkpoint

Eğitim sırasında kaydedilen model kopyası.

Head

Modelin son katmanı; sonucu (etiketi) üretir.

processing_class

Trainer'a hangi tokenizer'ın kullanılacağını söyler.

Loss

Modelin hata miktarı. Düşük olması iyidir.

Metrik

Modelin ne kadar iyi çalıştığını gösteren sayı (örn. accuracy).

Evaluation (Değerlendirme)

predict() komutu, validation set için modelin tahminlerini üretir. Her örnekte en yüksek skora sahip etiket modelin tahmini olur; bu tahminleri gerçek etiketlerle karşılaştırarak accuracy ve F1 değerlerini hesaplarız.

```
predictions = trainer.predict(tokenized_datasets["validation"])
# predictions.predictions: (408, 2) → 408 örnek x 2 logit
preds = np.argmax(predictions.predictions, axis=-1)

metric = evaluate.load("glue", "mrpc")
metric.compute(predictions=preds, references=predictions.label_ids)
# {'accuracy': 0.8578, 'f1': 0.8996}
```

Modelimiz validation set'te **%85,78 accuracy** ve **89,97 F1** değerine ulaştı; BERT makalesinde base model için F1 değeri 88,9 olarak verilmiştir.

compute_metrics ile otomatik evaluation (değerlendirme)

Bu hesaplamaları bir fonksiyonun içine koyup Trainer'a verirsek, model her epoch sonunda kendiliğinden değerlendirilir ve sonuçlar ekrana yazılır.

```
def compute_metrics(eval_preds):
    metric = evaluate.load("glue", "mrpc")
    logits, labels = eval_preds
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)

training_args = TrainingArguments("test-trainer", eval_strategy="epoch")
trainer = Trainer(..., compute_metrics=compute_metrics)
```

Not: Yeni bir eğitim için yeni TrainingArguments ve yeni model oluşturulmalıdır; yoksa eski model eğitime devam eder.

Advanced Features (İleri düzey özellikler)

Trainer'ın bazı hazır özellikleri, TrainingArguments içine tek bir ayar eklenerek açılabilir:

Özellik	Nasıl açılır?	Ne işe yarar?
Mixed precision	fp16=True	Eğitimi hızlandırır, belleği az kullanır.
Gradient accumulation	gradient_accumulation_steps=4	GPU belleği azken büyük batch etkisi verir.
Learning rate scheduler	lr_scheduler_type="cosine"	Learning rate'in zamanla nasıl azalacağını değiştirir.

Key Takeaways (Ana çıkarımlar)

- Trainer, eğitimin zor kısımlarının çoğunu bizim yerimize yapar.
- TrainingArguments ayarları, compute_metrics ölçümü belirler.
- fp16=True ve gradient accumulation, eğitimi daha hızlı ve verimli hale getirir.

Logits

Modelin her etiket için ham skoru. En büyüğü tahmindir.

argmax

En büyük değer in sırasını verir.

Accuracy

Doğru tahminlerin oranı.

F1

Precision ve recall'u birleştiren tek skor.

Epoch

Tüm eğitim verisinin modelden bir kez geçmesi.

Mixed precision

Daha az bellekli sayı formatıyla hızlı eğitim.

Gradient accumulation

Küçük batch'leri biriktirip büyük batch gibi eğitmek.

3/4 · Tam eğitim döngüsü

Bu bölümde aynı eğitimi Trainer kullanmadan, PyTorch ile kendi training loop'umuzu yazarak yapıyoruz. Bu yöntem, eğitimin her adımını kendimiz kontrol etmek istediğimizde işe yarar.

Preparation (Eğitime hazırlık)

Trainer'ın bizim yerimize yaptığı üç işi şimdi elle yapmamız gerekir: modelin kullanmadığı sütunları silmek, **label** sütununun adını **labels** yapmak ve veriyi PyTorch tensörlerine çevirmek.

```
tokenized_datasets = tokenized_datasets.remove_columns(
    ["sentence1", "sentence2", "idx"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch")
```

DataLoader

DataLoader, veriyi küçük gruplar (batch) halinde modele veren PyTorch aracıdır. Burada her grupta 8 örnek vardır; cümleleri aynı uzunluğa getirme (padding) işini ise daha önce hazırladığımız data_collator yapar.

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(tokenized_datasets["train"], shuffle=True,
    batch_size=8, collate_fn=data_collator)
eval_dataloader = DataLoader(tokenized_datasets["validation"],
    batch_size=8, collate_fn=data_collator)
```

Batch

Modele aynı anda verilen örnek grubu.

DataLoader

Veriyi batch batch modele veren araç.

Model, optimizer ve scheduler

AdamW, modelin ağırlıklarını güncelleyen optimizer'dır; **scheduler** ise learning rate'i 5e-5'ten başlayıp sıfıra kadar yavaş yavaş düşürür. Toplam adım sayısı epoch sayısı ile batch sayısının çarpımıdır: 3 epoch için 1377 adım.

```
from torch.optim import AdamW
from transformers import get_scheduler

optimizer = AdamW(model.parameters(), lr=5e-5)

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader) # 1377
lr_scheduler = get_scheduler("linear", optimizer=optimizer,
    num_warmup_steps=0, num_training_steps=num_training_steps)
```

Optimizer

Hataya bakıp modelin ağırlıklarını güncelleyen araç.

Scheduler

Learning rate'i eğitim boyunca ayarlar.

Learning rate

Modelin her adımda ne kadar değişeceği.

GPU kullanımı

GPU varsa modeli ve verileri GPU'ya taşırız, çünkü CPU'da eğitim birkaç dakika yerine saatler sürebilir.

```
device = torch.device("cuda") if torch.cuda.is_available() \
    else torch.device("cpu")
model.to(device)
```

Training Loop (Eğitim döngüsü)

Training loop'ta her batch için sırasıyla hatayı hesaplar, hatayı geriye doğru yayar, modeli günceller, learning rate'i ayarlar ve gradient'leri sıfırlarız.

```
progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()      # hatayı geriye yay
        optimizer.step()     # modeli güncelle
        lr_scheduler.step()  # learning rate'i ayarla
        optimizer.zero_grad() # gradient'leri sıfırla
        progress_bar.update(1)
```

Gradient

Hatayı azaltmak için ağırlıkların değişeceği yön.

Gradient clipping

Gradient'in aşırı büyümesini sınırlar.

Eğitimi geliştirmek için gradient clipping, mixed precision, gradient accumulation ve düzenli checkpoint kaydetme gibi yöntemler de eklenebilir.

Evaluation Loop (Değerlendirme döngüsü)

Evaluation loop'ta modelin tahminlerini **add_batch()** ile biriktirir, en sonda **compute()** ile accuracy ve F1 değerini hesaplarız. Bu sırada model öğrenmediği için **eval()** ve **no_grad()** kullanılır.

```
metric = evaluate.load("glue", "mrpc")
model.eval()
for batch in eval_dataloader:
    batch = {k: v.to(device) for k, v in batch.items()}
    with torch.no_grad():
        outputs = model(**batch)
        predictions = torch.argmax(outputs.logits, dim=-1)
        metric.add_batch(predictions=predictions,
                        references=batch["labels"])

metric.compute() # {'accuracy': 0.8431, 'f1': 0.8908}
```

Accelerate ile hızlandırma

Accelerate kütüphanesiyle aynı kodu birkaç küçük değişiklikle birden fazla GPU veya TPU üzerinde çalıştırabiliriz. Yapmamız gerekenler:

Accelerator() oluşturmak, nesneleri **prepare()** ile paketlemek, **.to(device)** satırlarını silmek ve **loss.backward()** yerine **accelerator.backward(loss)** yazmaktır.

```
from accelerate import Accelerator

accelerator = Accelerator()
train_dl, eval_dl, model, optimizer = accelerator.prepare(
    train_dataloader, eval_dataloader, model, optimizer)
...
accelerator.backward(loss) # loss.backward() yerine
```

Accelerate

Aynı kodu birden fazla GPU/TPU'da çalıştırır.

Distributed training

Eğitimi birden fazla GPU/TPU'ya bölmek.

Kodu **train.py** dosyasına koyduktan sonra önce **accelerate config** ile ayarları yapar, ardından **accelerate launch train.py** ile eğitimi başlatırız.

Next Steps (Sonraki adımlar)

- Modeli sadece accuracy ile değil, birden fazla metrikle değerlendirin.
- Hiperparametreleri Optuna veya Ray Tune gibi araçlarla sistemli biçimde deneyin.
- Eğitim sırasında learning curve'leri takip edin.
- Eğittiğiniz modeli Hugging Face Hub'da paylaşın.
- Büyük modeller için LoRA gibi daha verimli fine-tuning yöntemlerini deneyin.

3/5 · Öğrenme eğrilerini anlama

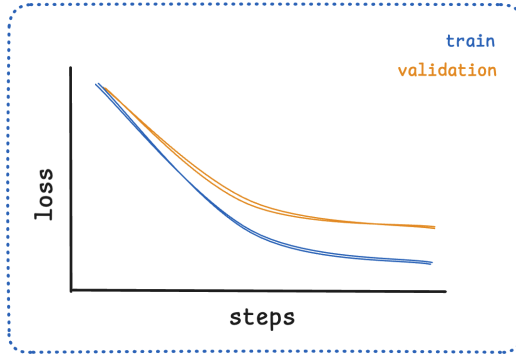
Learning curve'ler, modelin eğitim boyunca nasıl geliştiğini gösteren grafiklerdir. Bu grafiklere bakarak eğitimdeki sorunları, performans düşmeden önce fark edebiliriz.

Learning Curve (Öğrenme eğrisi) nedir?

- **Loss eğrisi:** Modelin hatasının eğitim boyunca nasıl değiştiğini gösterir.
- **Accuracy eğrisi:** Modelin doğru tahmin yüzdesinin eğitim boyunca nasıl değiştiğini gösterir.

Loss (Kayıp) eğrileri

Başarılı bir eğitimde loss önce yüksek başlar, eğitim ilerledikçe düşer ve sonunda düşük bir değerde sabitlenir; buna convergence denir.



Şekil 3: Başarılı bir eğitimde tipik loss eğrisi (şematik).

```
import wandb
wandb.init(project="transformer-fine-tuning")

training_args = TrainingArguments(
    output_dir="./results", eval_strategy="steps", eval_steps=50,
    logging_steps=10, num_train_epochs=3, report_to="wandb")
```

Eğrileri Weights & Biases'ta görmek için TrainingArguments içine **report_to="wandb"** yazmamız yeterlidir.

Learning curve

Eğitim boyunca loss veya accuracy'nin grafiği.

Loss

Modelin hata miktarı. Düşük olması iyidir.

Convergence

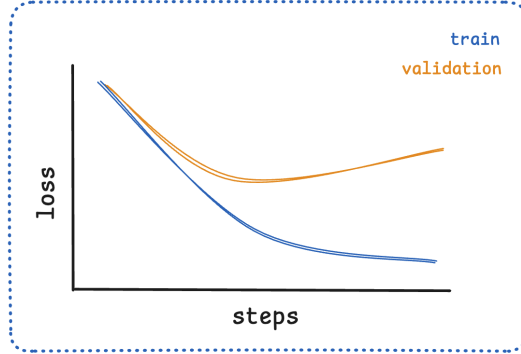
Loss'un düşük bir değerde sabitlenmesi.

Weights & Biases

Eğitim eğrilerini izleyen online araç.

Accuracy (Doğruluk) eğrileri

Accuracy genellikle zamanla artar, ancak düzgün bir çizgi değil basamaklar halinde yükselir; çünkü accuracy ancak tahmin doğru etikete geçtiğinde değişir.



Accuracy

Doğru tahminlerin oranı.

Plateau

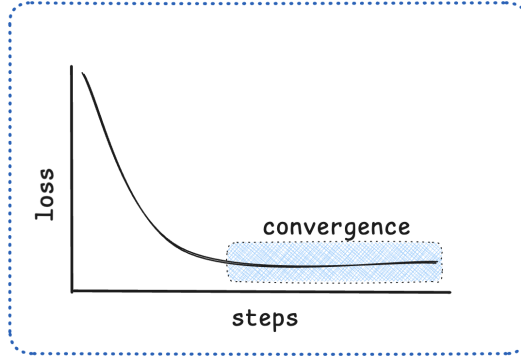
Eğrinin düz kaldığı bölüm.

Şekil 4: Loss pürüzsüz düşerken accuracy basamaklı artar (şematik).

Örnek: Bir köpek fotoğrafı (doğru cevap 1) için model önce 0,3, sonra 0,4 tahmin ediyor. İkisi de 0'a (yanlış sınıfa) yuvarlandığı için accuracy aynı kalır, ama 0,4 doğru cevaba daha yakın olduğu için loss düşer.

Convergence (Yakınsama)

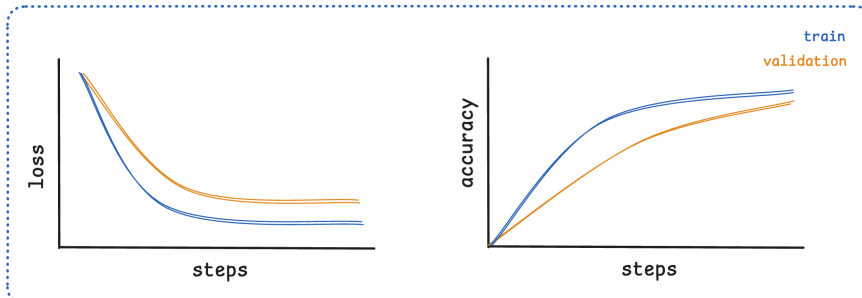
Loss ve accuracy eğrileri düzleştğinde model verideki örüntüleri öğrenmiş demektir ve yeni verilerde kullanılabilir. Hedefimiz, modeli her eğittiğimizde kararlı bir sonuca ulaşmaktır.



Şekil 5: Convergence (yakınsama): loss düşük bir değerde düzleşir.

Healthy Learning Curves (Sağlıklı öğrenme eğrileri)

Sağlıklı bir eğitimde loss düzgün biçimde azalır, training ve validation sonuçları birbirine yakın kalır ve eğriler sonunda düzleşir.



Validation set

Eğitimde kullanılmayan, kontrol için ayrılan veri.

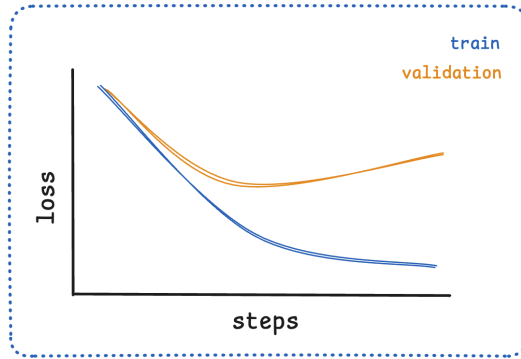
Şekil 6: Sağlıklı bir eğitimde loss (solda) ve accuracy (sağda) eğrileri (şematik).

During and After Training (Eğitim sırasında ve sonrasında)

Ne zaman?	Sorulacak sorular
Eğitim sırasında	Loss hâlâ düşüyor mu? Validation loss'u artmaya başladı mı? Eğri çok mu dalgalı?
Eğitim sonunda	Performans yeterli mi? Daha az epoch ile aynı sonuca ulaşılabilir miydi?

1. Overfitting (Ezberleme)

Overfitting'de model eğitim verisini ezberler; bu yüzden eğitim verisinde çok başarılı, daha önce görmediği verilerde ise başarısız olur.



Şekil 7: Overfitting. Training loss'u düşerken validation loss'u artar (şematik).

- **Belirti:** Training loss'u düşmeye devam ederken validation loss'u artar.
- **Çözüm:** Eğitimi erken durdurmak (early stopping), regularization eklemek, daha çeşitli veri kullanmak veya daha küçük bir model seçmek.

Aşağıdaki kod, validation loss'u art arda 3 evaluation'da iyileşmezse eğitimi otomatik olarak durdurur.

```
training_args = TrainingArguments(  
    output_dir="./results", eval_strategy="steps", eval_steps=100,  
    save_steps=100, load_best_model_at_end=True,  
    metric_for_best_model="eval_loss", greater_is_better=False,  
    num_train_epochs=10)  
  
trainer = Trainer(...,  
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)])
```

Overfitting

Ezberleme: eğitimde iyi, yeni veride kötü.

Regularization

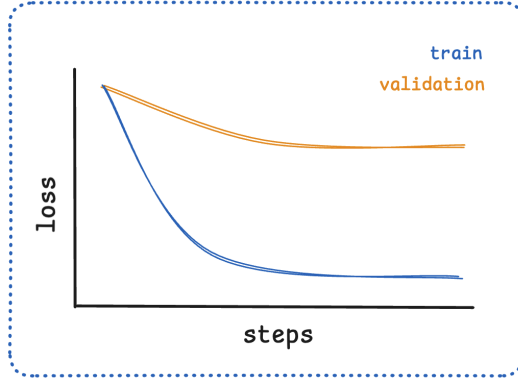
Ezberlemeyi azaltan teknikler (dropout, weight decay).

Early stopping

İyileşme durunca eğitimi erken bitirmek.

2. Underfitting (Yetersiz öğrenme)

Underfitting'de model, verideki örüntüleri öğrenemeyecek kadar basittir ya da yeterince eğitilmemiştir.



Şekil 8: Underfitting. Hem training hem validation'da loss yüksek kalır (şematik).

- **Belirti:** Hem training hem validation loss'u yüksek kalır.
- **Çözüm:** Daha büyük bir model kullanmak, daha uzun eğitmek, learning rate'i değiştirmek ve veri kalitesini kontrol etmek.

Örneğin epoch sayısını 5'ten 10'a çıkararak modele daha çok öğrenme fırsatı verebiliriz.

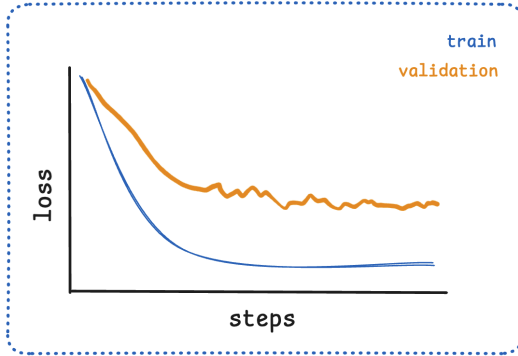
```
training_args = TrainingArguments(  
    output_dir="./results",  
    num_train_epochs=10) # 5 yerine 10
```

Underfitting

Yetersiz öğrenme: model verideki örüntüyü yakalayamaz.

3. Erratic Curves (Dengesiz eğriler)

Dengesiz eğrilerde loss veya accuracy sürekli inip çıkar; bu, modelin düzgün öğrenemediğini gösterir.



Şekil 9: Dengesiz eğriler. Loss ve accuracy net bir yön olmadan dalgalanır (şematik).

- **Belirti:** Eğriler net bir yön olmadan sürekli dalgalanır.
- **Çözüm:** Learning rate'i düşürmek, batch size'ı artırmak ve gradient clipping kullanmak.

Örneğin learning rate'i $1e-4$ 'ten $1e-5$ 'e düşürüp batch size'ı 16'dan 32'ye çıkarabiliriz.

```
training_args = TrainingArguments(  
    output_dir="./results",  
    learning_rate=1e-5, # düşür (1e-4 yerine)  
    per_device_train_batch_size=32) # artır (16 yerine)
```

Gradient clipping

Gradient'in aşırı büyümesini sınırlar.

Önemli çıkarımlar

- Learning curve'ler, eğitimin sağlıklı gidip gitmediğini anlamamızı sağlar.
- Overfitting'de training ve validation sonuçları birbirinden ayrılır.
- Underfitting'de model hem training set hem validation set'te başarısızdır.
- Early stopping ve regularization, sık görülen sorunların çoğunu çözer.

3/6 · Fine-tuning, Check! (Bölüm özeti)

Bu bölüm kapsamlıydı. İlk iki bölümde modelleri ve tokenizer'ları öğrenmiştik. Artık bunları modern en iyi pratiklerle kendi verimiz için nasıl ince ayarlayacağımızı da biliyoruz.

Bu bölümde neler yaptık?

Adım	Ne öğrendik?
1. Veri	Hub'daki veri setlerini ve modern veri işleme tekniklerini öğrendik.
2. Ön işleme	Veri setlerini verimli yükleyip işledik; dinamik padding ve data collator'ları kullandık.
3. Trainer	Üst düzey Trainer API ile ince ayar ve değerlendirmeyi yaptık.
4. Elle döngü	PyTorch ile baştan sona özel bir eğitim döngüsü yazdık.
5. Accelerate	Eğitim kodunu birden fazla GPU veya TPU'da sorunsuz çalışacak hale getirdik.
6. Optimizasyon	Karma hassasiyet ve gradient accumulation gibi modern teknikleri uyguladık.

Bu bölümde geliştirdiğimiz beceriler (veri ön işleme, eğitim ayarları, değerlendirme ve optimizasyon) her makine öğrenmesi projesi için temeldir. Metin sınıflandırma, adlandırılmış varlık tanıma (NER), soru yanıtlama ya da başka bir NLP görevi üzerinde çalışırken bu teknikler işinize yarayacaktır.

Başarı için ipuçları

- Özel eğitim döngüsü yazmadan önce her zaman **Trainer API** ile güçlü bir başlangıç noktası oluşturun.
- Daha iyi bir başlangıç için Hub'da **görevinize yakın** hazır modeller arayın.
- Eğitimi **uygun değerlendirme metrikleriyle** izleyin ve kontrol noktalarını (checkpoint) kaydetmeyi unutmayın.
- **Toplulukla paylaşın:** modellerinizi ve veri setlerinizi paylaşarak başkalarına yardım edin ve geri bildirim alın.

Sonraki adımlar

- Öğrendiğiniz teknikleri **kendi veri setinizde** deneyin.
- Hugging Face Hub'daki farklı **model mimarileriyle** deneyler yapın.
- Hugging Face topluluğuna katılıp projelerinizi paylaşın ve yardım alın.

Örnek: Bu yolculuğun sadece başlangıcı. Bir sonraki bölümde eğittiğimiz modelleri ve tokenizer'ları toplulukla nasıl paylaşacağımızı ve önceden eğitilmiş modeller ekosistemine nasıl katkı sağlayacağımızı göreceğiz.